Algoritmo di Bellman-Ford

Materia	Algoritmi
Anno	Secondo Anno
Data	@December 19, 2024
Studiato	Done

Proprietà dei cammini minimi e dei rilassamenti

Vale la **disuguaglianza triangolare:**

$$\forall (u,v) \in E, \delta(s,v) \leq \delta(s,u) + w(u,v)$$

che implica che **per ogni arco, il cammino da s a v dev'essere minore o uguale al cammino da s a u più il costo di (u,v).**

Vale inoltre la **upper-bound property**.

> *Per tutto il tempo della computazione, si ha:*
>
> $$v.d \geq \delta(s,v) \quad \forall v \in V \text{ e quando } v.d = \delta(s,v) \text{ esso non cambia più}$$

La terza proprietà è la **no-path property.**

> *Se non ci sono cammini da s a v, allora $\delta(s,v) = \infty$.*

La quarta proprietà è la **convergence property**:

> *Se $s \rightsquigarrow^p u \rightarrow v$ è un cammino minimo in G per qualche $u, v \in V$ e $u.d = \delta(s,u)$ in qualunque istante precedente a `relax(u,v,w)`, allora $v.d = \delta(s,v)$ in tutti gli istanti successivi.*

La quinta proprietà è la **predecessor-subgraph property.**

> *Una volta che $v.d = \delta(s,v)$ $\forall v \in V$, il grafo dei predecessori è un albero dei cammini minimi radicato ad s.*

L'ultima proprietà è la **path-relaxation property**.

> *Se $p = < v_0, ..., v_k >$ è un cammino minimo da $v_0 = s$ a v_k e se gli archi di p vengono rilassati nell'ordine:*
>
> $$(v_0, v_1), (v_1, v_2), ..., (v_{k-1}, v_k)$$

> *allora la stima $v_k.d = \delta(s, v_k)$ a prescindere dagli altri passi di rilassamento che possono occorrere, anche se inframezzati da rilassamenti degli archi di p.*

Correttezza dell'algoritmo di Bellman-Ford

Dimostreremo la correttezza l'algoritmo in diversi casi.
Cominciamo dal caso in cui non si siano cicli di peso negativo. Ignoriamo i vertici che non sono raggiungibili dalla sorgente. Per i vertici rimanenti, vogliamo che l'algoritmo restituisca il valore del cammino minimo.

Cominciamo dal seguente lemma.

> *Sia $G = (V, E)$ un grafo orientato, $s \in V$ un vertice sorgente, $w : E \to \mathbb{R}$ la funzione peso. Assumiamo che G **non contenga cicli di peso negativo** raggiungibili da s. Allora dopo $|V| - 1$ iterazioni del ciclo `for` di righe $2 - 4$ dell'algoritmo di Bellman-Ford vale:*
>
> $$v.d = \delta(s, v) \;\; \forall v \text{ raggiungibile da } s$$

Per dimostrare il lemma utilizziamo la path-relaxation property. Si consideri v un qualunque vertice raggiungibile da s e sia $p =< u_0, ..., u_k >$ un cammino minimo con $u_0 = s$ e $u_k = v$. Il cammino minimo è semplice, infatti se ci fosse un ciclo a peso non negativo non conviene ripeterlo. Poiché i cammini minimi sono semplici, p ha al più $|V| - 1$ archi, pertanto $k \leq |V| - 1$. Ognuna delle $|V| - 1$ iterazioni eseguite del ciclo `for` di righe $2 - 4$ rilassa tutti gli archi di E. Tra gli archi rilassati alla i-esima iterazione c'è sicuramente (u_{i-1}, u_i) e questo vale per ogni $i \in \{1, ..., k\}$. Per la path-relaxation property si ha che $v.d = v_k.d = \delta(s, v_k) = \delta(s, v)$.

Ne deduciamo il seguente corollario:

> *Sia $G = (V, E)$ un grafo orientato, $s \in V$ sorgente, $w : E \to \mathbb{R}$ funzione peso. Allora:*
>
> $$\forall v \in V, \exists \text{ un cammino da s a v } \iff \text{Bellman-Ford(G,s,w) termina con v.d} < \infty$$

Chiaramente se esiste un cammino da s a v, v è raggiungibile da s pertanto $v.d = \delta(s, v)$ che è più piccolo di infinito. Al contrario, se l'algoritmo termina con $v.d < \infty$, significa che la distanza è finita pertanto esiste un cammino.

Possiamo ora dimostrare la correttezza dell'algoritmo.

> *Supponiamo di eseguire Bellman-Ford su G, s, w dove $G = (V, E)$ grafo orientato, con $s \in V$ sorgente e $w : E \to \mathbb{R}$ funzione peso. Se non esistono cicli di peso negativo raggiungibili da s, allora:*
>
> - *l'algoritmo restituisce `true`;*
> - *$v.d = \delta(s, v)$ per ogni $v \in V$;*
> - *il sottografo dei predecessori G_π è un albero dei cammini minimi radicato ad s.*
>
> *Se esiste un ciclo di peso negativo raggiungibile da s allora l'algoritmo restituisce `false`.*

Supponiamo G non abbia cicli di peso negativo raggiungibili di s. Allora al termine dell'algoritmo $v.d = \delta(s, v)$ per ogni vertice del grafo. Infatti:

- se v è raggiungibile da s, vale per il lemma;
- se v non è ragginugibile da s, per il corollario $v.d = \infty$. Inoltre, $\delta(s, v) = \infty$ per la proprietà no-path quindi $v.d = \delta(s, v)$.

La predecessor-subgraph property assieme a quanto detto implica che G_π sia un albero di cammini minimi radicato ad s.
Osserviamo che al termine dell'esecuzione, per ogni
$(u, v) \in E$ vale:

$$v.d = \delta(s, v) \leq \delta(s, u) + w(u, v)$$

per la disuguaglianza triangolare. Quindi nessuno dei test a riga 6 ha successo e l'algoritmo restituisce `true`.

Supponiamo ora G abbia un ciclo di peso negativo raggiungibile da s. Sia esso $c =< v_0, ..., v_k >$, con $v_0 = v_k$. Allora:

$$w(c) = \sum_{i=1}^{k} w(v_{i-1}, v_i) < 0$$

Supponiamo per assurdo che l'algoritmo restituisca `true`. Ciò significa che nessuno dei test a riga 6 ha successo. Ossia, $v.d > u.d + w(u, v)$ non è mai soddisfatto. In particolare, ciò è vero per i vertici del ciclo, ossia:

$$\forall i \in \{1, ..., k\}, v_i.d \leq v_{i-1}.d + w(v_{i-1}, v_i)$$

Ma allora sommando membro a membro per $i = 1, ..., k$ si ha che:

$$\sum_{i=1}^{k} v_i.d \leq \sum_{i=1}^{k} [v_{i-1}.d + w(v_{i-1}, v_i)] =$$
$$= \sum_{i=1}^{k} v_{i-1}.d + \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

Poiché $v_k = v_0$, ogni vertice di c è contato esattamente una sola volta nelle due somme, e si ha che:

$$\sum_{i=1}^{k} v_i.d = \sum_{i=1}^{k} v_{i-1}.d$$

Inoltre, queste somme **non divergono**, in quanto il ciclo è raggiungibile da s, e per il corollario si verifica che $v_i.d \neq \infty$. Sottraendo ad ambo i membri $\sum_{i=1}^{k} v_{i-1}.d$:

$$0 \leq \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

che è un assurdo in quanto abbiamo detto che $w(c) < 0$ e quindi significa che l'algoritmo deve restituire `false`.

Introduzione all'algoritmo di Dijkstra

L'algoritmo di Dijkstra risolve il Single-Source Shortest Path Problem nel caso in cui $w : E \to \mathbb{R}_0^+$. Con una buona implementazione, il tempo computazionale dell'algoritmo di Dijkstra è inferiore a quello di Bellman-Ford.

Possiamo pensare all'algoritmo di Dijkstra come a una generalizzazione della ricerca BFS ai grafi pesati. Esso mantiene un insieme S di vertici i cui pesi dei cammini minimi da s sorgente sono già stati determinati. L'algoritmo seleziona ripetutamente il vertice $u \in V \setminus S$ con la stima del cammino minimo più piccola, aggiunge u ad S e rilassa tutti gli archi uscenti da u.

```
Dijkstra(G, s, w):
    Initialize-Single-Source
    S = empty
    Q = empty
    for each vertex v in V do
        insert(Q, v)
    while Q is not empty do
        u = extract-min(Q)   // u tale che u.d sia minimo
        S = union(S, {u})
        for each v in Adj(u) do
            relax(u, v, w)
            if the call of relax decreased v.d
                decrease-key(Q, v, v.d)
```